PROCESS MANAGEMENT

Condensed Study Notes

Generated: 2026-05-25 00:09 UTC

PROCESSMANAGEMENT

Process Management is the operating system's role in handling the execution of programs. A 'process' is a program in execution, possessing its own memory space and resources.

Key aspects include:

- Process Creation: Starting new processes.
- Process Termination: Ending running processes.
- Process Scheduling: Deciding which process gets to use the CPU next and for how long.
- Process Synchronization: Coordinating multiple processes that need to share resources.
- Inter-Process Communication (IPC): Allowing processes to exchange information.

PROCESSCONCEPT

A process is an instance of a program that is currently executing.

Historically, batch systems used the term 'jobs' to refer to running programs. This terminology is still prevalent, and 'job' and 'process' are often used interchangeably in modern contexts, particularly in areas like job scheduling.

THEPROCESS

A process's memory is divided into four main sections:

- **Text Section:** Contains the compiled program code, loaded from storage when the program starts.
- **Data Section:** Stores global and static variables, initialized before program execution begins.
- **Heap:** Used for dynamic memory allocation, managed by functions like `new` or `malloc`.
- **Stack:** Used for local variables and function return values. Space is reserved upon declaration and freed when variables go out of scope. Stack management can be language-specific.

Notably, the stack and heap start at opposite ends of the process's free memory space and grow towards each other. If they meet, it signals an error (e.g., stack overflow or insufficient memory for allocation).

When processes are moved out of and back into memory, crucial information like the program counter and register values must also be saved and restored.

PROCESSCREATION

Process creation is the initial step for a process to begin execution. It is achieved through the `fork()` system call.

Four primary events can cause a process to be created:

- System initialization
- A user request to create a new process
- Initiation of a batch job
- Execution of a process-creation system call by a running process.

A system call is a mechanism that provides the interface between a process and the operating system, allowing user programs to request services.

When a process creates another process, the creator is called the parent, and the created process is called the child.

Each process is assigned a unique integer identifier called its Process Identifier (PID). The Parent Process Identifier (PPID) is also stored for each process.

On typical UNIX systems, the process scheduler (`sched`) has PID 0. At system startup, `sched` launches `init` (PID 1), which then becomes the ultimate parent of all other processes by launching system daemons and user logins.

PROCESSTERMINATION

Processes, like programs, eventually end. Termination can occur voluntarily, involuntarily, or be initiated by other processes or the system.

Voluntary Termination:

- **Normal Exit:** A process finishes its assigned task and signals completion to the operating system using a system call like `exit()` (UNIX) or `ExitProcess` (Windows).
- **Error Exit:** A process encounters a fatal error (e.g., trying to compile a non-existent file, executing an illegal instruction, or dividing by zero) and terminates itself.

Involuntary Termination:

- **Killed by Another Process:** One process can issue a system call (e.g., `KILL` in UNIX, `TerminateProcess` in Win32) to terminate another.
- **System Termination:** The operating system may terminate a process due to various reasons:
- The parent process no longer needs its children.
- The parent process exits, and the system decides to terminate its children.
- A `KILL` command or unhandled process interrupt is received.
- The system cannot provide necessary resources.

When a process terminates, its system resources (like open files) are freed. The termination status and execution times are passed to the parent process if it's waiting. If a process's parent has exited and the child is still running, the child may become an orphan and be inherited by the `init` process (PID 1).

Zombie Process: A process that has terminated but whose termination status has not yet been collected by its parent is called a zombie. These are eventually inherited by `init` and cleaned up.

PROCESSSTATE

As a process executes, its current activity defines its state. A process can be in one of the following states:

- **New**: The process is in the initial stage of creation.
- **Running**: The CPU is actively executing the process's instructions.
- **Ready**: The process has all necessary resources to run but is waiting for the CPU to become available. It may enter this state after starting or be interrupted while running.
- **Waiting**: The process is temporarily unable to run, awaiting an event or resource. This could be keyboard input, disk access, inter-process messages, a timer, or a child process completing.
- **Terminated**: The process has finished execution. It is moved to this state and waits for removal from main memory.

Diagramofprocessstate

The **Process Control Block (PCB)** is a data structure managed by the operating system that stores all information needed to maintain track of a process. Each process has a unique integer **Process ID (PID)** to identify its PCB.

The PCB architecture varies by OS, but typically includes:

- **Process State**: The current status of the process (e.g., Running, Waiting).
- **Process ID and Parent Process ID**: Unique identifiers for the process and the process that created it.
- **Program Counter**: Stores the address of the next instruction to be executed; saved/restored when processes are swapped.
- **CPU Registers**: Values in CPU registers (accumulators, index registers, stack pointers, etc.) that must be saved/restored for execution.
- **Process Privileges**: Permissions to access system resources.
- **CPU-Scheduling Information**: Includes priority and pointers to scheduling queues.
- **Memory-Management Information**: Details like page tables or segment tables, and memory limitations.
- **I/O Status Information**: Lists allocated devices, open file tables, etc.
- **Accounting Information**: Tracks CPU time used, account numbers, and limits.

Processcontrolblock(PCB) OPERATIONSONPROCESSES

The operating system allows users to perform operations on processes. These operations include:

- **Process Creation**: Initiating new processes.
- **Process Scheduling/Dispatching**: Deciding which process gets to use the CPU and for how long.

AtreeofprocessesonatypicalLinuxsystem

Child processes may share some resources with their parent. Resource limits can be applied to child processes to prevent excessive consumption.

After a parent process creates a child, it has two main options:

1. **Wait for child termination:** The parent calls `wait()`, blocking until the child finishes. This is typical for UNIX shells before issuing a new prompt.

2. **Run concurrently:** The parent continues execution without waiting for the child. This is used for background tasks in shells or in parallel processing scenarios where the parent might perform some work before waiting for children.

Preemption

Preemption is the operating system's capability to interrupt a currently running task and switch to a higher-priority task.

This interruption can occur when the resource being scheduled is the processor or I/O.

ProcessScheduling

Scheduling, also known as dispatching, is the OS action of moving a process from the Ready state to the Running state.

This occurs when:

- Free resources become available.
- A higher-priority process arrives than the one currently executing.

Blocking

Blocking occurs when the system must wait for an input/output (I/O) operation to complete.

When a process is blocked, the operating system moves it to the Waiting state.

A blocked process cannot execute until the shared resource it needs becomes available.

Shared resources include:

- CPU
- Network and network interfaces
- Memory
- Disk

INTER-PROCESS COMMUNICATION

Processes running concurrently can be either independent or cooperating.

Independent Processes: Cannot affect or be affected by other processes.

Cooperating Processes: Can affect or be affected by other processes. Any process sharing data is a cooperating process. Reasons for allowing cooperating processes include:

- Information Sharing: Multiple processes accessing the same file (e.g., pipelines).
- Convenience: A single user multitasking (editing, compiling, printing, running code in different windows).
- Modularity: Breaking a system into cooperating modules (e.g., client-server databases).
- Computation Speedup: Solving problems faster by dividing them into sub-tasks for simultaneous solution, especially with multiple processors.

Cooperating processes require Inter-Process Communication (IPC), typically via two methods:

1. Shared Memory Systems:
   - Faster once set up, as no system calls are needed; access is at normal memory speeds.
   - More complicated to set up.
   - Does not work well across multiple computers.
   - Preferable for large amounts of data shared quickly on the same computer.
2. Message Passing Systems:
   - Requires system calls for every message transfer, making it slower.
   - Simpler to set up.
   - Works well across multiple computers.
   - Preferable for small amounts/frequency of data transfers, or when multiple computers are involved.